BoilerBridge

CS307
Design Document

Team 26:
Logan Wilkinson
Xavion Marable
Emilio Guzman
Shreyansh Tehanguria
Shashank Mukkera
Vanessa Gomez Miselem

Index

1. Purpose

1.1 Problem Statement

A known issue for college students is finding group travel that correlates with their interests as well as their financial constraints. While there are other travel platforms like **Expedia™** and **TripAdvisor™,** most of the existing platforms cater mostly to families or business travel. These platforms also do not have tools like group based travel options or budget management, which are super important for students that will have to travel on limited funds. With our travel planner, our users will be able to build itineraries, discover new destinations, and manage shared costs in one app, allowing students to focus on the experience of the trip instead of the stress of planning.

1.2 Functional Requirements

1. User Account
   As a user,
   a. I would like to be able to create an account.
   b. I would like to be able to login and manage my account settings.
   c. I would like to be able to edit my profile details, like name, school, location, and profile picture.
   d. I would like to be able to reset my password via email if I forget it.
   e. I would like to be able to reset my password by text message for faster access.

2. Travel Groups
   As a user,
   a. I would like to be able to create travel groups.
   b. I would like to be able to manage my travel group.
   c. I would like to be able to leave my travel group.
   d. I would like to be able to access a shared calendar for my travel group.

   As a group leader,
   a. I would like to be able to transfer my leader's permission to another member.
   b. I would like to be able to remove specific people from the travel group.

3. Itinerary Creation
   As a user,
   a. I would like to be able to input my travel preferences.
   b. I would like to create a budget for the trip.
   c. I would like to be able to specify transportation preferences.
   d. I would like to be able to add activities/locations as must haves.
   e. I would like to be able to add activities/locations to avoid.
   f. I would like to be able to generate a random location based on budget.
   g. I would like to be able to see recommendations/extra info for activities.

4. Itinerary Interaction
   As a user,
   a. I would like to be able to see reviews for places people have travelled.
   b. I would like to be able to see a summary of reviews for activities.
   c. I would like to be able to visit sites for more information about activities.
   d. I would like to be able to go to activity booking sites via external links.
   e. I would like to be able to find temporary bag storage.
   f. I would like to be able to have an offline itinerary view.
   g. I would like to be able to see a visual map representation of the itinerary.
   h. I would like to be able to have an offline map view.
   i. I would like to be able to create reminders for my travel plans.
   j. I would like to be able to access an SOS button that shows emergency numbers.
   k. I would like to be able to write reviews for places I've visited.

5. Itinerary Modification
   As a user,
   a. I would like to be able to generate alternative plans.

b. I would like to be able to make changes to only one part of an itinerary.
c. I would like to be able to click a button to regenerate selected options.
d. I would like to be able to click a button to remove options.
e. I would like to be able to vote on specific activity options created by the itinerary generator.
f. I would like to be able to retrieve previous itineraries.
g. I would like to be able to post itineraries visible to all app users.
h. I would like to be able to share itineraries via link.
i. I would like to be able to export itineraries to my preferred calendar.

6. Itinerary Budget
   As a user,
   a. I would like to be able to post a shared cost to the group ledger.
   b. I would like to be able to split the costs with my group.
   c. I would like to be able to view a list of what everyone owes at the end of the trip.
   d. I would like to be able to request payment from specific group members for a shared cost.
   e. I would like to be able to send a payment confirmation notice to a member after paying them back.

7. Communication
   As a user,
   a. I would like to be able to connect with new people by finding and adding them as friends.
   b. I would like to be able to create a shared photo album for trips.
   c. I would like to be able to send text notifications to my group.
   d. I would like to be able to send email notifications to my group.
   e. I would like to be able to send in-app notifications to my group.
   f. I would like to be able to receive text notifications about my travel.
   g. I would like to be able to receive email notifications about my travel.
   h. I would like to be able to receive in-app notifications about my travel.

i. I would like to be able to share my real-time location with added friends.

1.3 Non-Functional Requirements

1.3.1 Architecture and Performance

We are developing BoilerBridge as a web application using **Next.js**. With this framework, we will combine our frontend and backend into a single codebase, making it much easier to manage user accounts and itinerary updates in real time. We will use **MongoDB** as our NoSQL database to handle the relationships between users, groups, and itineraries. This architecture allows for efficient storage of scraped travel data, allowing the system to reuse some of that information for the future. To provide a high quality experience to our user, all group updates and messages should synchronize across all user devices in **500ms**.

1.3.2 Security

Security is vital for BoilerBridge since it will handle user data and group transaction information. We will implement a secure login system with **OAuth 2.0** to manage the user accounts. There will be roles and permission systems to make sure that users can only access the itineraries and messaging boards for the groups that they have joined. Any requests to the backend will be authenticated to prevent any unauthorized access or abuse of our service.

1.3.3 Usability

The interface should be easy to navigate so that the planning process is as stress free as possible. Since the platform has many tools like messaging, calendars, and expense tracking, the interface has to be well designed to make sure that students can coordinate quickly, and on all screen sizes. The goal is to have a central platform where all group members can see updates, like any wishlist items or itinerary changes, in the same place.

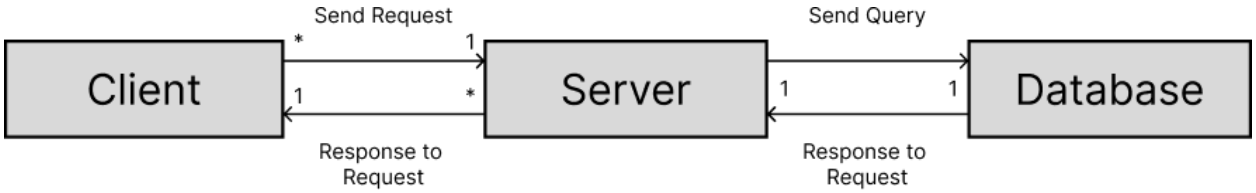
1.3.4 Deployment

Since our frontend and backend are integrated with Next.js, we can deploy our entire project as one unit. This makes it easier to test any new features like our cost-splitting logic or the AI itinerary generator. We will host the app in a **Node.js** environment that supports the real time collaboration our users need.

2. Design Outline

2.1 High Level Overview

This project will be a web application that allows users to create travel itineraries based on preferences in a group or individually, send messages and split costs in said groups, and find new people to travel with. This application will use a client-server model in which one server will handle simultaneous access from a large number of clients using the Next.js framework in typescript. The server will accept requests from clients to access, store, or update data in the database, and return data to the client(s) accordingly.

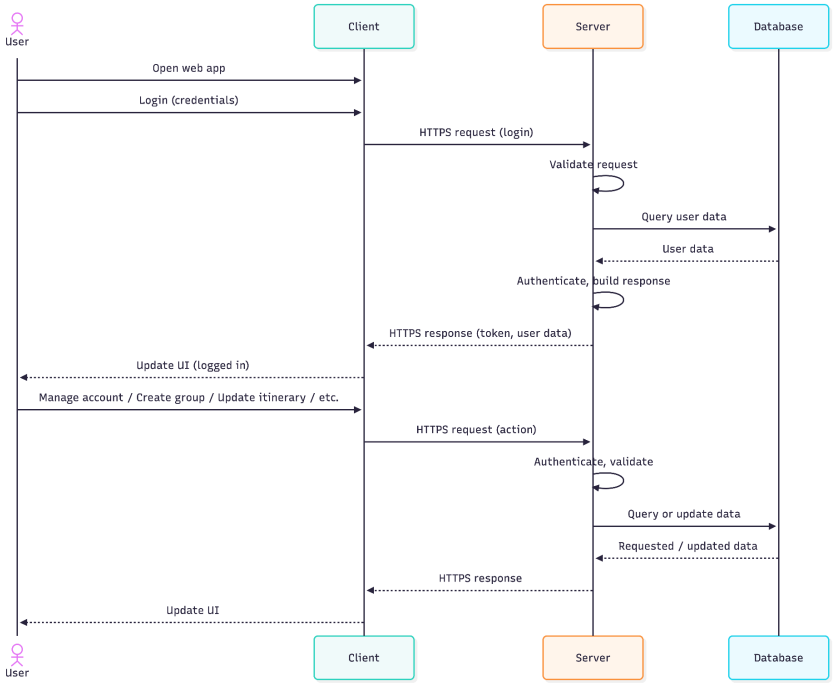


1. **Client**
    a. Client provides a user an interface for our system
    b. Client sends HTTPS requests to the server
    c. Client receives and interprets HTTPS responses from the server and updates the user interface accordingly.
2. **Server**
    a. Server receives and handles HTTPS requests from clients
    b. Server validates requests and queries the database to add, modify or retrieve data on client demand
    c. Server generates appropriate HTTPS responses to client requests and returns appropriate data
    d. Server authenticates users before processing protected routes
    e. Server acts as the central authority for group membership and permissions
    f. Server performs itinerary generation and activity validation before storing data
    g. Server ensures multiple users cannot overwrite shared data simultaneously
    h. Server communicates with external services such as AI itinerary generation and location data APIs

3. **Database**
    a. A non-relational database to store all data used by the application, including but not limited to: user data, travel itineraries, and previously scraped travel data
    b. Database responds to queries from the server and send the requested data back to the server
    c. Database is never accessed directly by the client
    d. Database stores persistent state of groups, activities, and expenses
    e. Database allows flexible storage of varying activity fields from external APIs
    f. Database supports concurrent reads and writes from multiple users through the server

2.2 Sequence of Events Overview

The sequence diagram below shows typical interactions between clients, server, and database. The sequence is initiated by a user opening the web app. As the user logs in, the client sends a request to the server. The server then handles this request and sends a query to the database. The server then receives said data from the database and constructs it's reply to the client that made the request. After logging in, the client send further requests to the server for other actions, like managing account data, creating, updating and deleting travel itineraries and creating travel groups. To respond to these requests the server send queries to the database to acquire or update the requested data. The database responds to the server with updated or requested data and returns the result back to the client.



3. Design Issues

3.1 Functional Issues

- **Concurrent Modification Conflicts in Real Time Group Editing**
  - **Problem:** When multiple users simultaneously edit itineraries, vote or update expenses, data conflict may occur and overwrite important information
  - **Options:**
    i. Pessimistic locking: lock the itinerary or group data while one users edits it so others cannot modify it until the lock is released.
    ii. Optimistic concurrency control: Attach version number to shared data. If user submits changes and the version has changed, the system rejects the update and asks the client to refresh
    iii. Event Sourcing: Instead of modifying shared data directly, store each action (ex: “addExpense”, “vote”) and reconstruct state from these actions
  - **Best Option:** Optimistic concurrency allows real-time collaboration without blocking users. It prevents overwrites while maintaining good user experience. It is more scalable than locking mechanisms and simpler than event sourcing.

- **Validity of AI-Generated Itineraries and Place Verification**
  - **Problem:** The LLM may generate activities that do not exist, are duplicated or fail to verification through the Places API.
  - **Options:**
    i. Strict Validation: Reject the itinerary if any activity fails verification.
    ii. Partial Acceptance with Regeneration: Accept verified activities and regenerate only failed ones
    iii. Pre-Validated Activity Pool: Fetch real places first and then allow the LLM to organize and schedule from verified candidates.
  - **Best Option:** The best option is to use option iii and option ii as a fallback. Using verified place data before AI scheduling reduces hallucinations and invalid entries. Regeneration only failed activities improves user experience and reliability without discarding valid results.
- **Server-Authoritative Expense Ledger Integrity**
  - **Problem:** Client side calculation of expense splits can lead to inconsistencies, rounding errors or manipulation.
  - **Options:**
    i. Client-Side Calculation: The client computes splits and sends final values to the server.
    ii. Server-Authoritative Calculation: The client sends raw inputs (payer, amount, participants), and the server computes splits and updates balances.

iii. Hybrid Verification: Client computes splits; server recomputes and verifies consistency before saving.

- **Best Option:** Server-Authoritative Ledger is the best option. Financial calculations must be trusted and consistent. Centralizing calculations on the server ensures integrity, prevents tampering and avoids discrepancies across devices.

- **Fair and Stable Preference Aggregation Mechanism**
  - **Problem:** Simple majority voting may produce unfair or unstable itinerary results, especially when preferences conflict
  - **Options:**
    i. Majority Voting: Activities with the most vots are selected.
    ii. Weighted Voting: Assign different weights to users (ex: group creator has more influence)
    iii. Multi-Criteria Scoring Model: Score activities based on interest, budget, compatibility, availability, among others.
  - **Best Option:** The best option is Multi-Criteria Scoring. A scoring model produces more balanced and explainable results. It reduces bias and ensures the itinerary reflects both preferences and constraints.
- **Group Membership Consistency During Active Planning**
  - **Problem:** Adding or removing members during planning can create inconsistencies in voting and expense splits.
  - **Options:**
    i. Retroactive Recalculation: Recompute all previous votes and expenses whenever membership changes.
    ii. Snapshot Membership Model: Each expense records participants at creation time. Future changes do not affect past records.
    iii. Freeze Membership After Planning Begins: Disallow membership changes once itinerary generation starts.
  - **Best Option:** The best option is the Snapshot Model. The snapshot model preserves historical accuracy without expensive recalculations. It prevents financial disputes and maintains system stability.
- **Time Zone Consistency and Calendar Integration Accuracy**
  - **Problem:** Incorrect time zone handling can shift events, cause overlaps or misalign schedules across users.
  - **Options:**
    i. Store All times in UTC: Convert to local time only when displaying or exporting.
    ii. Store in Destination Time Zone: Use the trip location time zone for all itinerary events.
    iii. Manual Confirmation at Export: Ask users to confirm time zone before calendar export.
  - **Best Option:** UTC storage prevents daylight savings and conversion errors. It ensures consistent time handling across users and external calendar systems.

3.2 Non-Functional Issues

- **Scalability of AI and Places API Integration**
  - **Problem:** High demand for itinerary generation may exceed API rate limits and increase latency.
  - **Options:**
    - On-Demand Calls Only: Call API's for every request without optimization.
    - Caching Frequent Requests: Store results for common destinations and reuse them.
    - Queue-Based Processing with Rate Limiting: Place requests in a controlled queue to manage API usage.
  - **Best Option:** The best option is a combination of Caching frequent requests and queue-based processing. Caching reduces repeated calls, and queuing controls request bursts. Together they improve scalability and cost efficiency.
- **Data Privacy and Protection of Financial Information**
  - **Problem:** The system stores personal travel plans and shared financial data that must be protected.
  - **Options:**
    - HTTPS Only: Encrypt data in transit only.
    - Encryption at Rest and HTTPS: Encrypt stored database data and use secure transport.
    - Role-Based Access Control and Encryption: Limit access permissions and encrypt stored data.
  - **Best Option:** Role-Based Access Control combined with encryption ensures that only authorized users can access sensitive information while protecting data both in transit and at rest.
- **Reliability Under External API Failures**
  - **Problem:** The system depends on external services that may fail or change unexpectedly.
  - **Options:**
    - Hard Failure Response: Display an error if APIs fail.
    - Fallback Templates: Provide a generic itinerary when AI fails.
    - Cached Historical Results: Reuse stored itineraries if APIs are unavailable.
  - **Best Option:** The best option is a combination of Fallback Templates and Cached historical results. Fallback mechanisms maintain system functionality during outages and improve perceived reliability
- **Real-Time Performance and Responsiveness**
  - **Problem:** Users expect fast updates across devices, but polling or heavy database operations may introduce delays.
  - **Options:**
    - Periodic Polling: Clients check for updates at fixed intervals.

        - WebSocket-Based Real-Time Updates: Server pushes updates instantly to clients.
        - Hybrid Approach: Use WebSockets for chat and polling for less important data.
    - **Best Option:** The best option is the selective websocket usage. WebSockets provide low-latency synchronization, improving collaboration and user experience.
- **Cost Control and API Budget Sustainability**
    - **Problem:** Frequent AI and Places API calls may lead to unpredictable operational costs.
    - **Options:**
        - Unlimited Usage: No restrictions on API calls.
        - Rate Limiting per User/Group: Restrict frequency of AI requests.
        - Usage Quotas or Token Caps: Limit total AI usage per user.
    - **Best Option:** The best option is a combination of rate limiting and usage quotas. Rate limits and quotas prevent cost escalation and protect the system from abuse.
- **Maintainability and Extensibility of System Architecture**
    - **Problem:** Tightly coupled integration of AI, Places, ledger and messagin may increase technical debt.
    - **Options:**
        - Monolithic Architecture: All logic in one tightly connected system.
        - Layered Architecture with Service Abstractions: Separate concerts (API layer, service layer, provider interfaces).
        - Full Microservices Architecture: Split components into independently deployable services.
    - **Best Option:** The best option is layered architecture with abstractions. Layered architecture reduced coupling while avoiding the operational complexity of microservices. It supports future model swaps and feature expansion.

4. Design Details

4.1 Class Design

This section outlines the data models used in our Next.js backend and MongoDB database. These classes handle the logic for user management, group coordination, and itinerary generation.

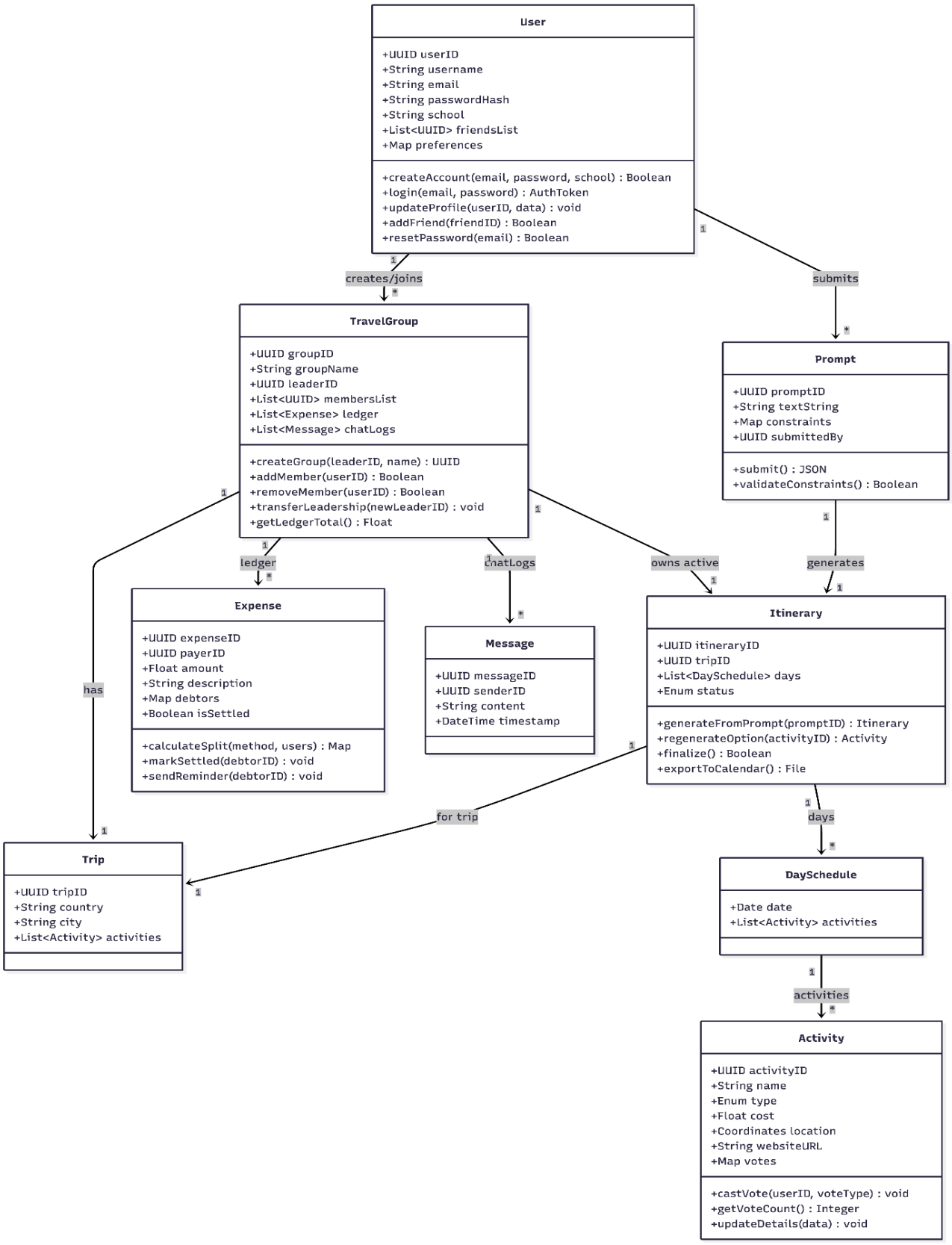
- **User**
    - Description: The main account holder. Using OAuth 2.0 for authentication.

    - Interactions: A user creates/joins a `TravelGroup` and owns an `Itinerary`.
    - Attributes:
        - `userID: UUID`
        - `username: String`
        - `email: String`
        - `passwordHash: String`
        - `school: String`
        - `friendsList: List<UUID>`
        - `preferences: Map<String, Boolean>`
    - Operations:
        - `createAccount(email, password, school): Boolean`
        - `login(email, password): AuthToken`
        - `updateProfile(userID, data): void`
        - `addFriend(friendID): Boolean`
        - `resetPassword(email): Boolean`
- **TravelGroup**
    - Description: Manages the state of a specific trip, including members and chat.
    - Attributes:
        - `groupID: UUID`
        - `groupName: String`
        - `leaderID: UUID`
        - `membersList: List<UUID>`
        - `ledger: List<Expense>`
        - `chatLogs: List<Message>`
    - Interactions: Links a list of `User` objects to a `Trip`. Contains the shared ledger of `Expense` objects.
    - Operations:
        - `createGroup(leaderID, name): UUID`
        - `addMember(userID): Boolean`
        - `removeMember(userID): Boolean`

        - `transferLeadership(newLeaderID): void`
        - `getLedgerTotal(): Float`
- **Itinerary**
    - Description: The schedule generated by the LLM. It will organize activities into a timeline.
    - Interactions: Generated from a `Prompt`. It will hold a list of Activity objects.
    - Attributes:
        - `itineraryID: UUID`
        - `tripID: UUID`
        - `days: List<DaySchedule>`
        - `status: Enum {DRAFT, VOTING, CONFIRMED}`
    - Operations:
        - `generateFromPrompt(promptID): Itinerary`
        - `regenerateOption(activityID): Activity`
        - `finalize(): Boolean`
        - `exportToCalendar(): File`
- **DaySchedule**
    - Description**:** A helper class that organizes activities by specific dates within the Itinerary.
    - Interactions**:** Contained within an Itinerary. Holds a list of Activity objects for that specific date.
    - Attributes**:**
        - `date: Date`
        - `activities: List<Activity>`
- **Activity**
    - Description: A specific event of location within a trip. It will include pricing, addresses, and a voting system for group decision making.
    - Interactions: Users will vote on these. Data will be populated using the WebScraper service or manual user input.
    - Attributes:
        - `activityID: UUID`
        - `name: String`
        - `type: Enum {FOOD, SIGHTSEEING, EVENT}`
        - `cost: Float`

        - `location: Coordinates`
        - `websiteURL: String`
        - `votes: Map<UUID, VoteType>`
    - Operations:
        - `castVote(userID, voteType): void`
        - `getVoteCount(): Integer`
        - `updateDetails(data): void`
- **Expense**
    - Description: A financial transaction used for cost-splitting. It will track who paid and how much other group members owe in the ledger.
    - Interactions: Linked to the TravelGroup ledger. Updates the debt status of any involved users.
    - Attributes:
        - `expenseID: UUID`
        - `payerID: UUID`
        - `amount: Float`
        - `description: String`
        - `debtors: Map<UUID, Float>`
        - `isSettled: Boolean`
    - Operations:
        - `calculateSplit(method, users): Map<UUID, Float>`
        - `markSettled(debtorID): void`
        - `sendReminder(debtorID): void`
- **Prompt**
    - Description: The input interface for the AI generator that captures any user constraints like budget and duration.
    - Interactions: Sent to the backend to trigger the LLM itinerary generation logic.
    - Attributes:
        - `promptID: UUID`
        - `textString: String`
        - `constraints: Map<String, String>` (Budget, Dates)
        - `submittedBy: UUID`

    - Operations:
        - `submit(): JSON`
        - `validateConstraints(): Boolean`
- **Message**
    - Description**:** Represents a single chat message sent within the group's communication board.
    - Interactions**:** Stored in the TravelGroup's chatLogs.
    - Attributes**:**
        - `messageID: UUID`
        - `senderID: UUID`
        - `content: String`
        - `timestamp: DateTime`


User
+UUID userID
+String username
+String email
+String passwordHash
+String school
+List<UUID> friendsList
+Map preferences
+createAccount(email, password, school) : Boolean
+login(email, password) : AuthToken
+updateProfile(userID, data) : void
+addFriend(friendID) : Boolean
+resetPassword(email) : Boolean
creates/joins
submits
TravelGroup
+UUID groupID
+String groupName
+UUID leaderID
+List<UUID> membersList
+List<Expense> ledger
+List<Message> chatLogs
+createGroup(leaderID, name) : UUID
+addMember(userID) : Boolean
+removeMember(userID) : Boolean
+transferLeadership(newLeaderID) : void
+getLedgerTotal() : Float
Prompt
+UUID promptID
+String textString
+Map constraints
+UUID submittedBy
+submit() : JSON
+validateConstraints() : Boolean
ledger
chatLogs
owns active
generates
has
Expense
+UUID expenseID
+UUID payerID
+Float amount
+String description
+Map debtors
+Boolean isSettled
+calculateSplit(method, users) : Map
+markSettled(debtorID) : void
+sendReminder(debtorID) : void
Message
+UUID messageID
+UUID senderID
+String content
+DateTime timestamp
Itinerary
+UUID itineraryID
+UUID tripID
+List<DaySchedule> days
+Enum status
+generateFromPrompt(promptID) : Itinerary
+regenerateOption(activityID) : Activity
+finalize() : Boolean
+exportToCalendar() : File
for trip
days
Trip
+UUID tripID
+String country
+String city
+List<Activity> activities
DaySchedule
+Date date
+List<Activity> activities
activities
Activity
+UUID activityID
+String name
+Enum type
+Float cost
+Coordinates location
+String websiteURL
+Map votes
+castVote(userID, voteType) : void
+getVoteCount() : Integer
+updateDetails(data) : void

4.1.1 Class Responsibilities and Interactions

The classes in BoilerBridge are designed to separate user management, trip coordination, and itinerary generation into independent but connected components. Each class owns its own data while referencing others through identifiers to avoid tight coupling.

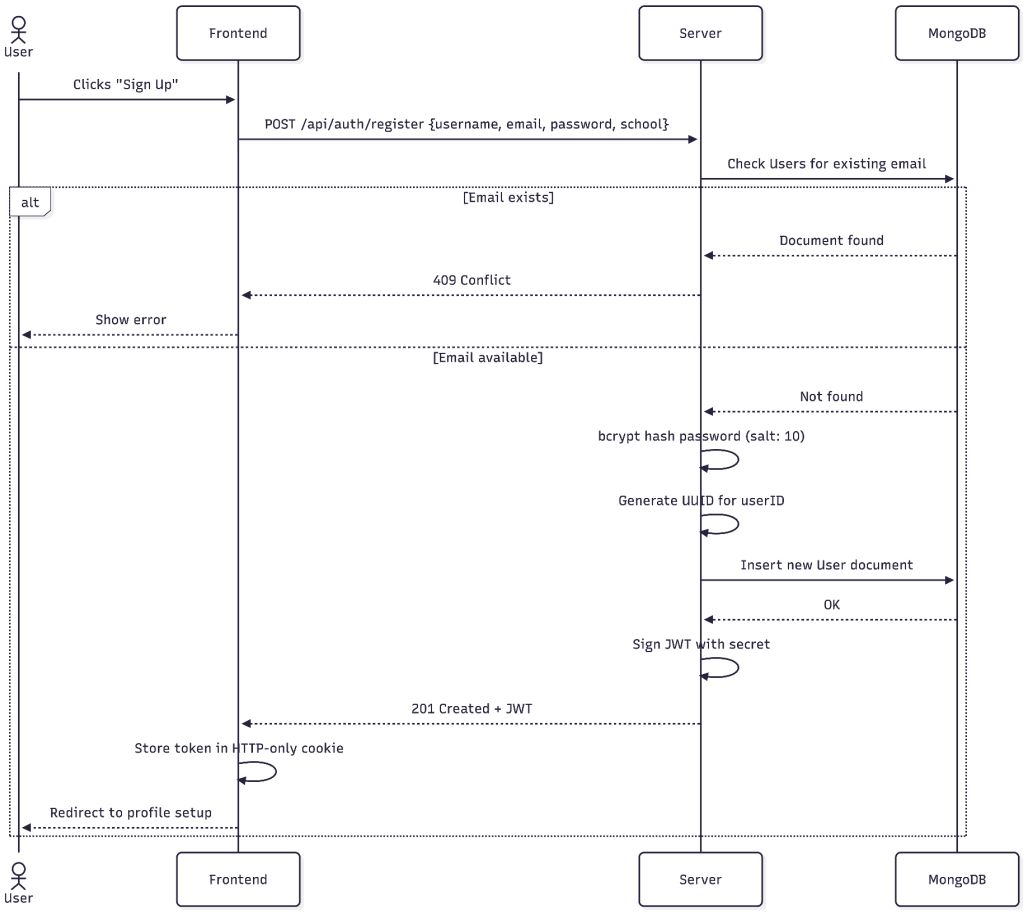
- **User ↔ TravelGroup**
  A User can create or join multiple TravelGroups. The TravelGroup stores only the userID references instead of duplicating user data. This allows user profile updates to automatically reflect across all groups. Only the group leader is allowed to modify group membership or transfer leadership.
- **TravelGroup ↔ Expense**
  The TravelGroup maintains a shared financial ledger. Whenever a user adds a cost, an Expense object is created and appended to the group ledger. The TravelGroup verifies membership before accepting the expense and updates the outstanding balances of each member.
- **TravelGroup ↔ Itinerary**
  Each TravelGroup owns exactly one active itinerary at a time. The itinerary is generated based on a Prompt submitted by the group leader and remains editable while in DRAFT state. Once finalized, only minor updates such as voting and regeneration of individual activities are allowed.
- **Itinerary ↔ Activity**
  An Itinerary is composed of multiple DaySchedules, each containing Activity objects. Activities store voting data for group decision making. Users never modify the itinerary directly; they interact through voting or regeneration requests, which preserves consistency of the schedule.
- **Prompt ↔ Itinerary (AI Generation)**
  The Prompt acts as the input contract between users and the AI generator. After validation, it is sent to the LLM service which returns structured activities. The backend then verifies the activities using the WebScraper/Places service before creating Activity objects inside the Itinerary.
- **Activity ↔ User (Voting System)**
  Each Activity maintains a vote map linking userIDs to vote types. This ensures one vote per user and allows the system to dynamically rank activities based on group preferences without rewriting the itinerary.

This structure ensures that user data, trip coordination, financial tracking, and AI generation remain modular while still enabling real-time collaboration.

4.2 Sequence of Events

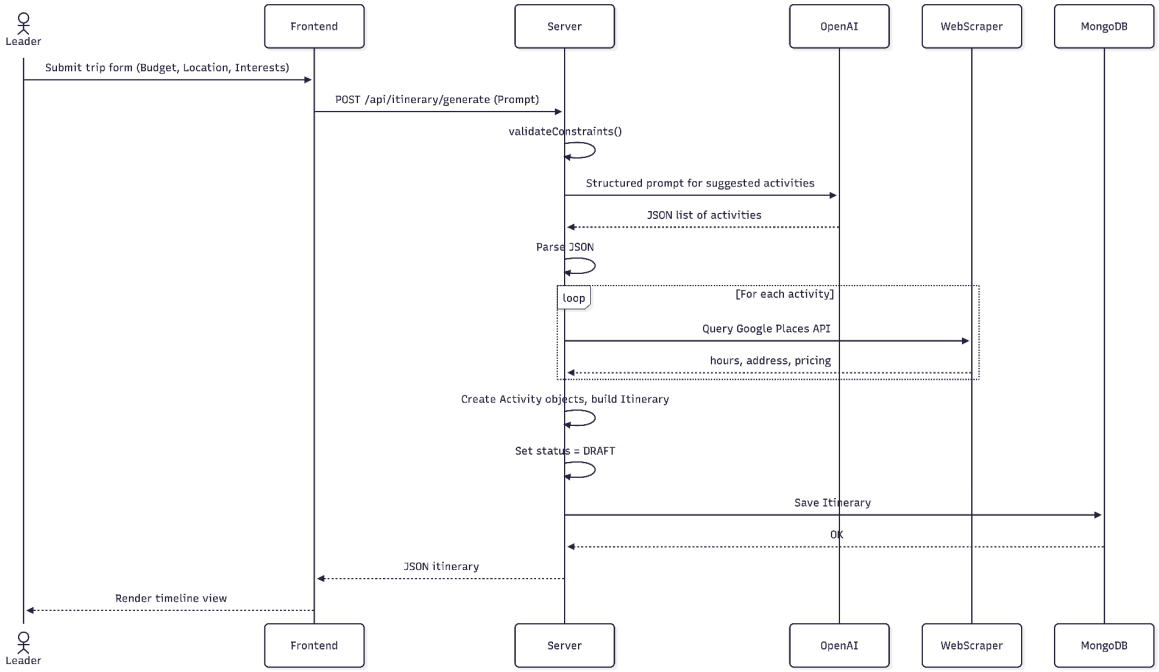
4.2.1 User Registration & Authentication

1. **User Action**: User clicks “Sign Up” on the landing page.
2. **Client Request:** Frontend sends a `POST` request to `/api/auth/register` with payload `{username, email, password, school}`.
3. **Server Validation:** Server checks the `Users` collection in MongoDB for existing emails. Returns `409 Conflict` if found.
4. **Security:** Server hashes the password using **bcrypt** (salt rounds: 10) and generates a UUID.
5. **Database Write:** Server inserts the new document into the `Users` collection.
6. **Token:** Server generates a **JWT** signed with the backend secret.
7. **Response:** Server returns `201 Created` with the token. Client stores it in an HTTP-only cookie and redirects to profile setup.



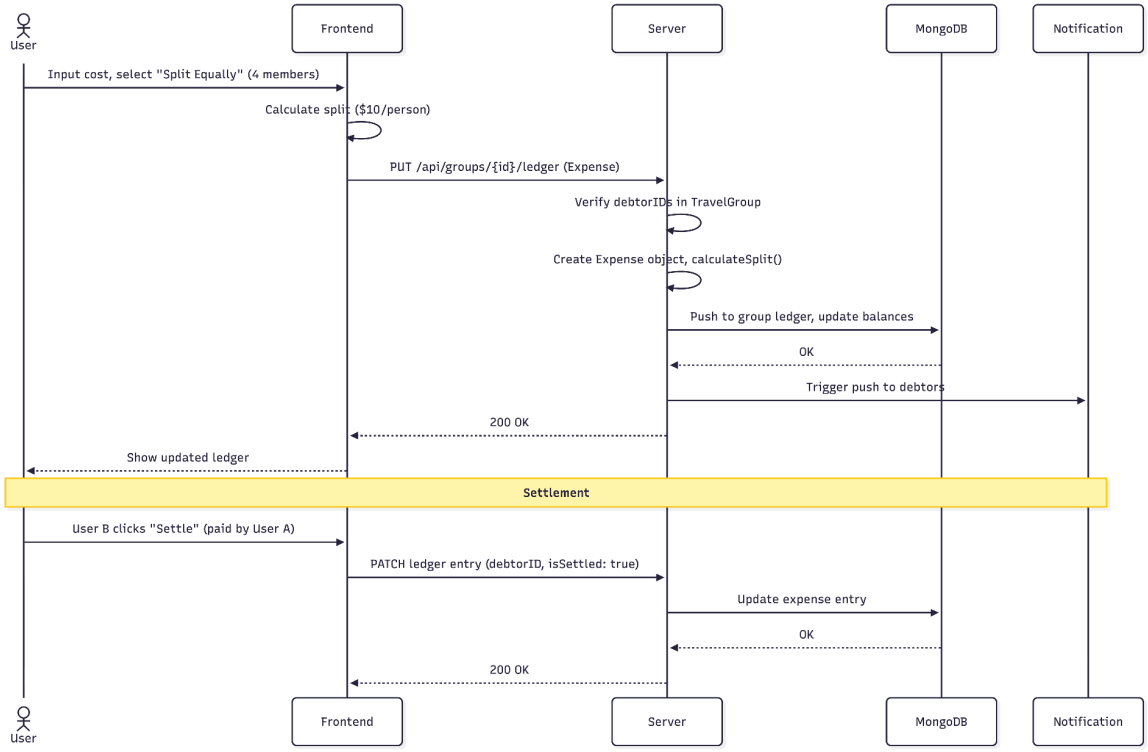
4.2.2 Creating a Group & Itinerary

1. **User Action:** Group Leader submits the trip form with constraints: `Budget`, `Location`, and `Interests`.
2. **Client Request:** Frontend sends `POST` to `/api/itinerary/generate` with the `Prompt` object.
3. **LLM Processing:** Server sends a structured prompt to the **OpenAI API**. API returns a JSON list of suggested activities.
4. **Data Verification:** Server parses the JSON. `WebScraper` service queries **Google Places API** for each activity to get real-time hours, address, and pricing.
5. **Object Creation:** Server instantiates `Activity` objects for valid results and bundles them into an `Itinerary`.
6. **Database Write:** Server saves the `Itinerary` with status `DRAFT`.
7. **Display:** Server returns the JSON. Frontend renders the timeline view for the Leader to review.


Leader
Frontend
Server
OpenAI
WebScraper
MongoDB
Submit trip form (Budget, Location, Interests)
POST /api/itinerary/generate (Prompt)
validateConstraints()
Structured prompt for suggested activities
JSON list of activities
Parse JSON
loop
[For each activity]
Query Google Places API
hours, address, pricing
Create Activity objects, build Itinerary
Set status = DRAFT
Save Itinerary
OK
JSON itinerary
Render timeline view
Leader
Frontend
Server
OpenAI
WebScraper
MongoDB

4.2.3 Budget & Splitting

1. **User Action:** User inputs a shared cost (e.g., Uber) and selects "Split Equally" between 4 members.
2. **Client Request:** Frontend calculates the split ($10/person) and sends `PUT` to `/api/groups/{id}/ledger`.
3. **Server Logic:** Server verifies all `debtorIDs` are in the `TravelGroup`. Instantiates a new `Expense` object.
4. **Database Update:** Server pushes the object to the group `ledger` array and updates the `balance` field for each user.
5. **Notification:** Server triggers push notifications to the users who owe money.
6. **Settlement:** User A pays User B. User B clicks "Settle." Server updates that specific debt entry to `isSettled: true`.



4.3 Database Design

We are using a NoSQL database (MongoDB) to handle travel itineraries where data structures for activities may vary. Below are the JSON schemas for our primary collections.

User Collection Stores authentication data, profile details, and social graph connections.

```
{
  "userID": "UUID",
  "username": "String",
  "email": "String",
  "passwordHash": "String",
  "school": "String",
  "friendsList": ["UUID"],
  "preferences": {
    "interests": ["String"],
    "budget_tier": "Integer"
  }
}
```

Groups Collection Stores group metadata and embeds the Ledger(Expenses) for quick access to financial totals.

```
{
  "groupID": "UUID",
  "groupName": "String",
  "leaderID": "UUID",
  "membersList": ["UUID"],
  "ledger": [
    {
      "expenseID": "UUID",
      "payerID": "UUID",
      "amount": "Float",
      "description": "String",
      "debtors": { "userID": "Float" },
      "isSettled": "Boolean"
    }
  ],
  "chatLogs": [
    {
```

```
      "messageID": "UUID",
      "senderID": "UUID",
      "content": "String",
      "timestamp": "Date"
    }
  ]
}
```

Itineraries Collection Linked to Groups. Stores the structure of Trips -> Days -> Activities

```
{
  "itineraryID": "UUID",
  "groupID": "UUID",
  "status": "String",
  "days": [
    {
      "date": "Date",
      "activities": [
        {
          "activityID": "UUID",
          "name": "String",
          "location": { "lat": "Float", "lng": "Float" },
          "cost": "Float",
          "votes": { "userID": "VoteType" }
        }
      ]
    }
  ]
}
```

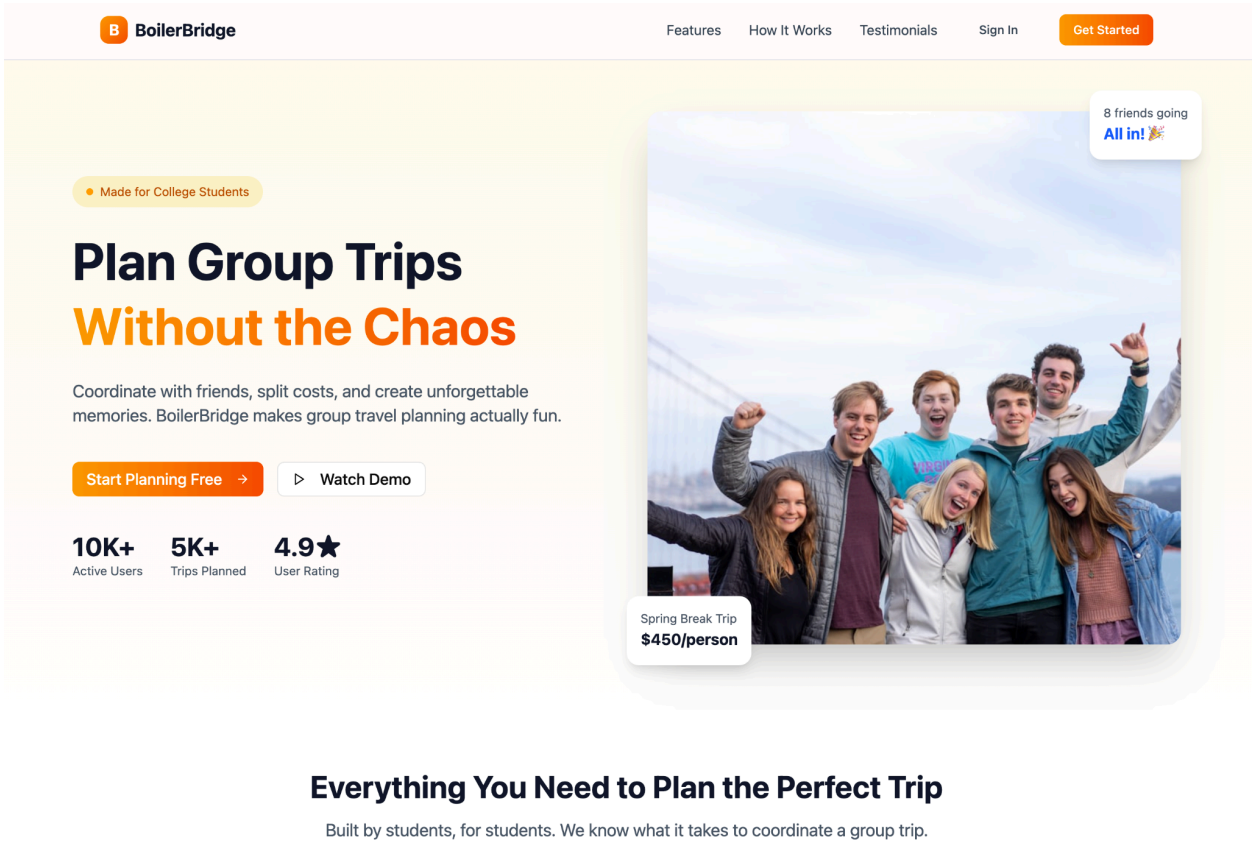
4.4 API Routes

The API Routes will handle the communication between the client and server in the client-server architecture. Each route will have a resource that will be provided by the server or updates by the client with HTTP requests.

- **`POST /api/auth/register`** – Creates a new user document in the database.
- **`GET /api/user/{id}/groups`** – Returns a list of all groups the user is a member of.

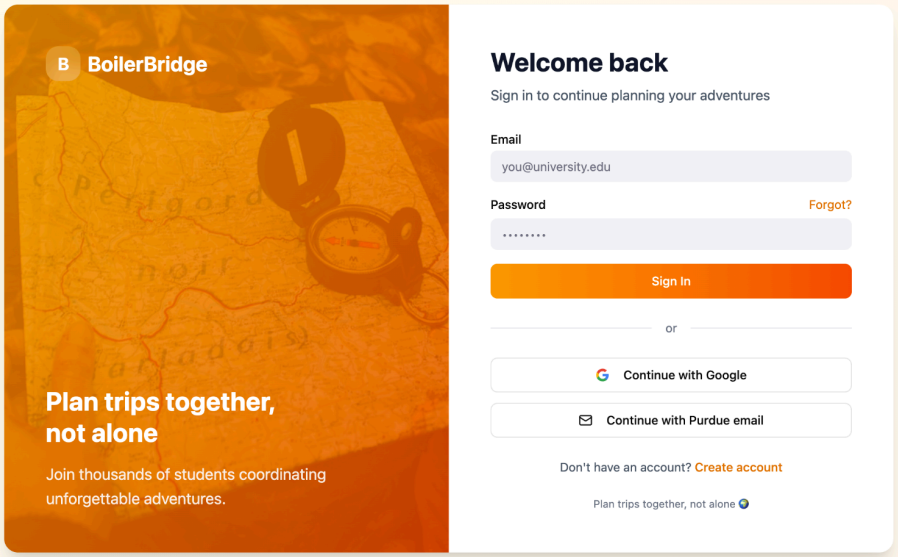
- `POST /api/groups/create` – Initializes a new travel group in MongoDB.
- `POST /api/itinerary/generate` – Triggers the LLM and WebScraper to return a new activity list.
- `PUT /api/groups/{id}/ledger` – Posts a new shared cost to the group expense ledger.
- `GET /api/groups/{id}/chat` – Retrieves the message history for a group's communication hub.

4.5 UI Mockup

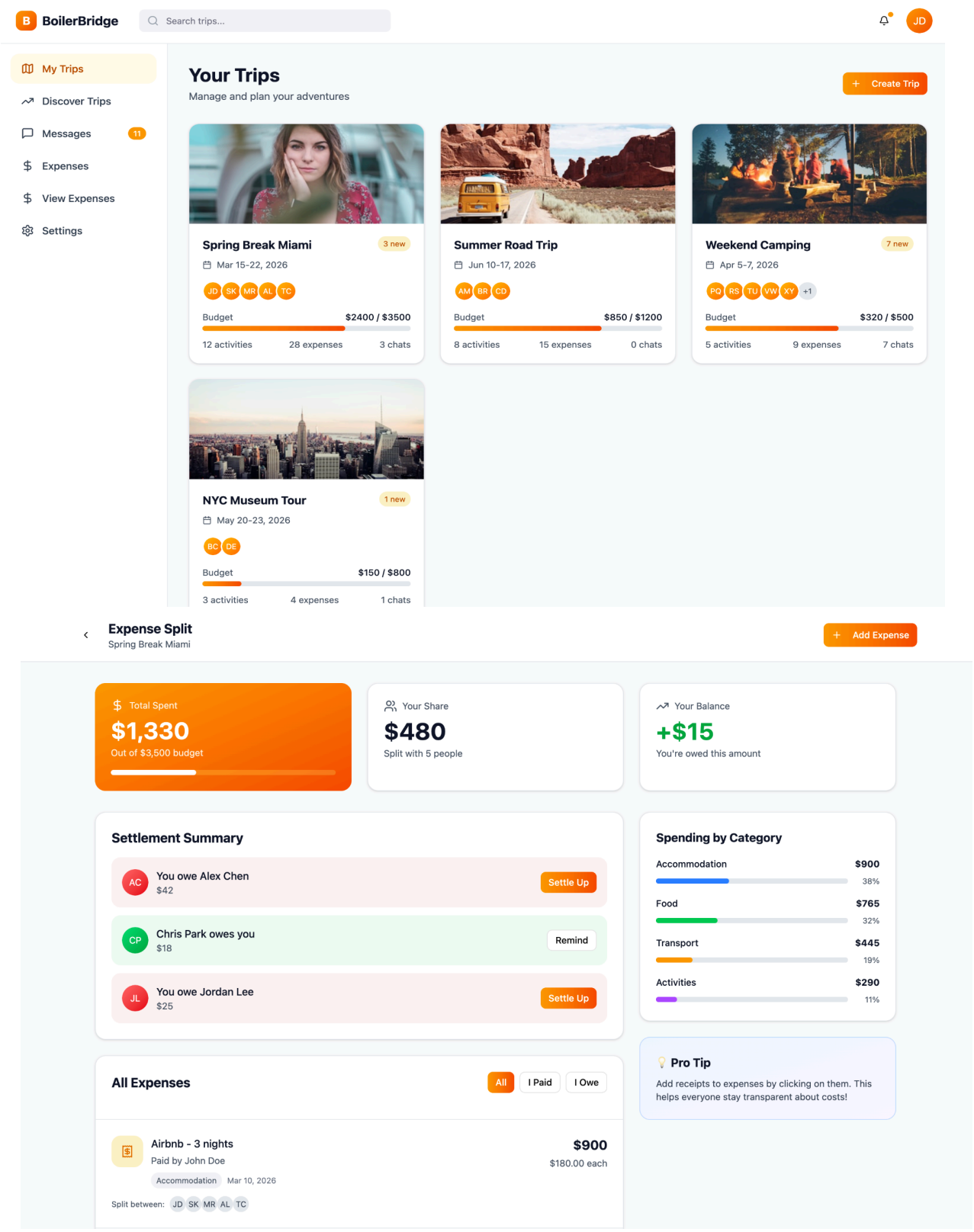


The landing page introduces the BoilerBridge platform and communicates the purpose of the application: simplifying group travel planning. Users can quickly understand the core features such as itinerary generation, cost splitting, and collaboration before creating an account. The page focuses on clarity and quick

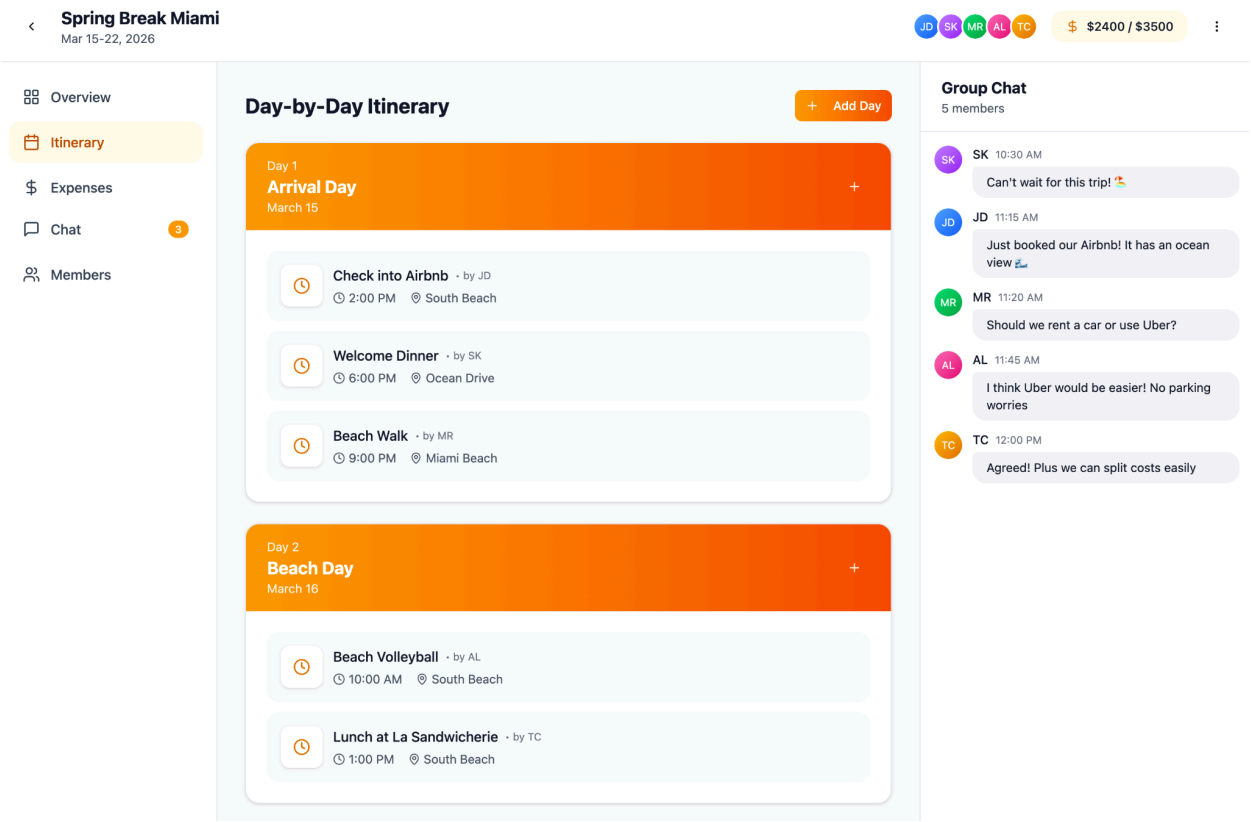
onboarding for first-time visitors. This page is the entry point of the application and encourages users to sign up or log in to begin planning trips.



The sign-in page allows existing users to securely access their accounts using email and password authentication. It also provides alternative authentication options and a link to create a new account. The design prioritizes simplicity and fast access.



The expense page tracks shared costs and displays balances between group members. The system calculates how much each member owes and shows a clear summary of payments and settlements.



This page provides a day-by-day itinerary alongside a group chat. Users can coordinate in real time while reviewing planned activities. Combining communication and planning reduces the need for external messaging apps.